

Lab 2: The EC-1 Microprocessor

Microprocessors (010153521) | Semester 1/2026

Duration: 3 hours (in-lab) | **Topic:** ISA, datapath construction, and hardwired FSM control-unit design in Logisim-Evolution

Objectives

- Translate a datapath block diagram (registers, incrementer/decrementer, muxes, tri-state buffer) into working Logisim-Evolution components.
- Design a hardwired FSM control unit from an ISA and the fetch/decode/execute instruction cycle: choose states, draw the state diagram, encode it, and derive Boolean next-state and output equations.
- Hand-assemble an EC-1 program, load it into ROM, and verify its execution cycle by cycle.

Quick Reference: EC-1 ISA

EC-1 is an 8-bit-data, 4-bit-address machine: 16 ROM locations, each holding one 8-bit instruction word ($IR[7:5]$ = opcode, $IR[4:0]$ = operand).

Instruction	Mnemonic	Encoding	Operation
Input	IN A	011 xxxxx	$A \leftarrow \text{Input}$
Output	OUT A	100 xxxxx	$\text{Output} \leftarrow A$
Decrement	DEC A	101 xxxxx	$A \leftarrow A - 1$
Jump if Not Zero	JNZ addr	110 xaaaa	if $A \neq 0$: $PC \leftarrow aaaa$
Halt	HALT	111 xxxxx	stop execution

Opcodes 000, 001, 010 are unused (no-op: return to FETCH). x = don't-care; aaaa = a 4-bit ROM address.

EC-1 Datapath

Figure 1 is the conceptual datapath (Lecture 10, Figure 10-1). Every control signal (**IRload**, **PCload**, **JNZmux**, **INmux**, **Aload**, **OutE**, **Reset**, **Clock**) enters from the left; you will design the logic that generates them in Exercise 5.

Figure 2 is the reference Logisim-Evolution circuit – build *exactly this*, component for component, using the same signal names.

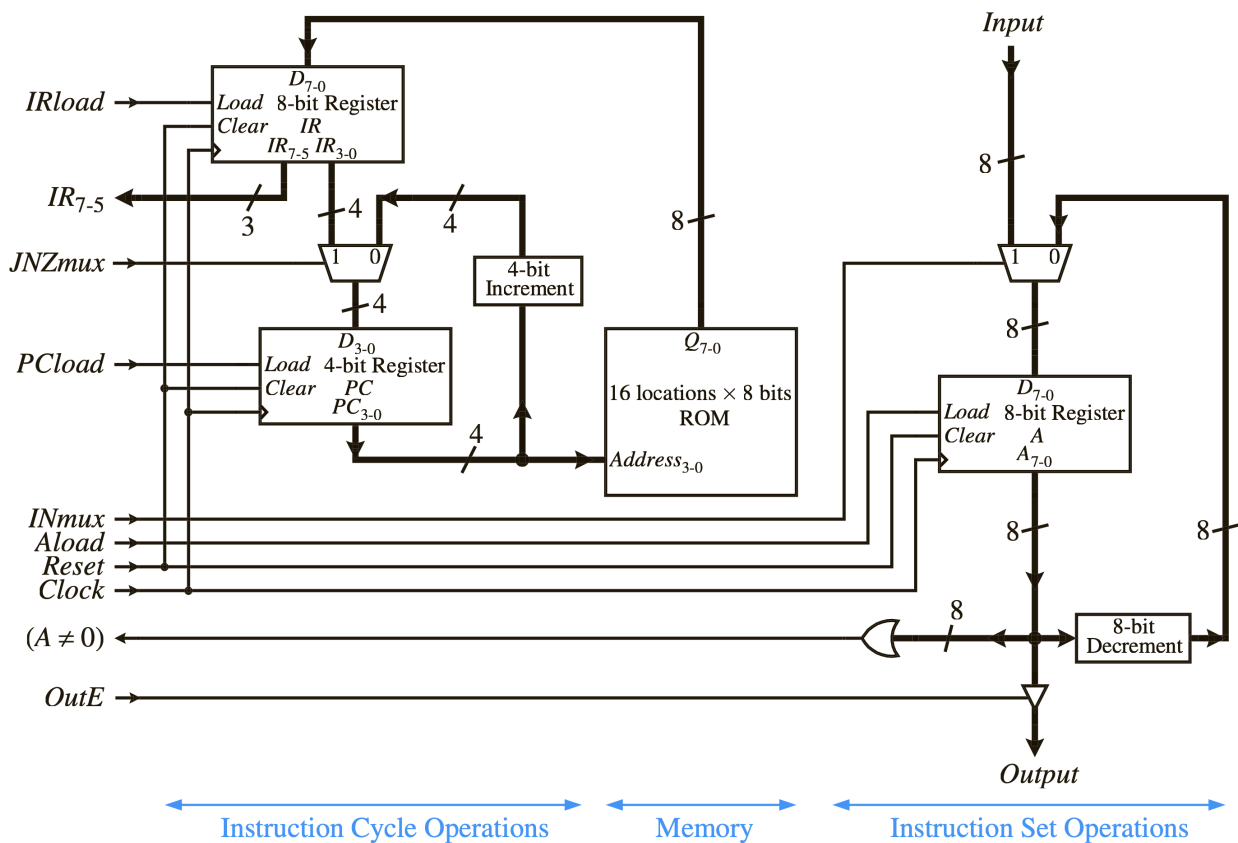


Figure 1: EC-1 datapath (Lecture 10, Figure 10-1). Left: Instruction Cycle Operations (*IR*, *JNZmux*, *PC*, the 4-bit Incrementer). Centre: the 16×8 program **ROM**, addressed *directly* by *PC* – there is no separate address register. Right: Instruction Set Operations (*INmux*, the Accumulator *A*, the 8-bit Decrementer, the zero-check producing $(A \neq 0)$, and the tri-state output buffer gated by *OutE*).

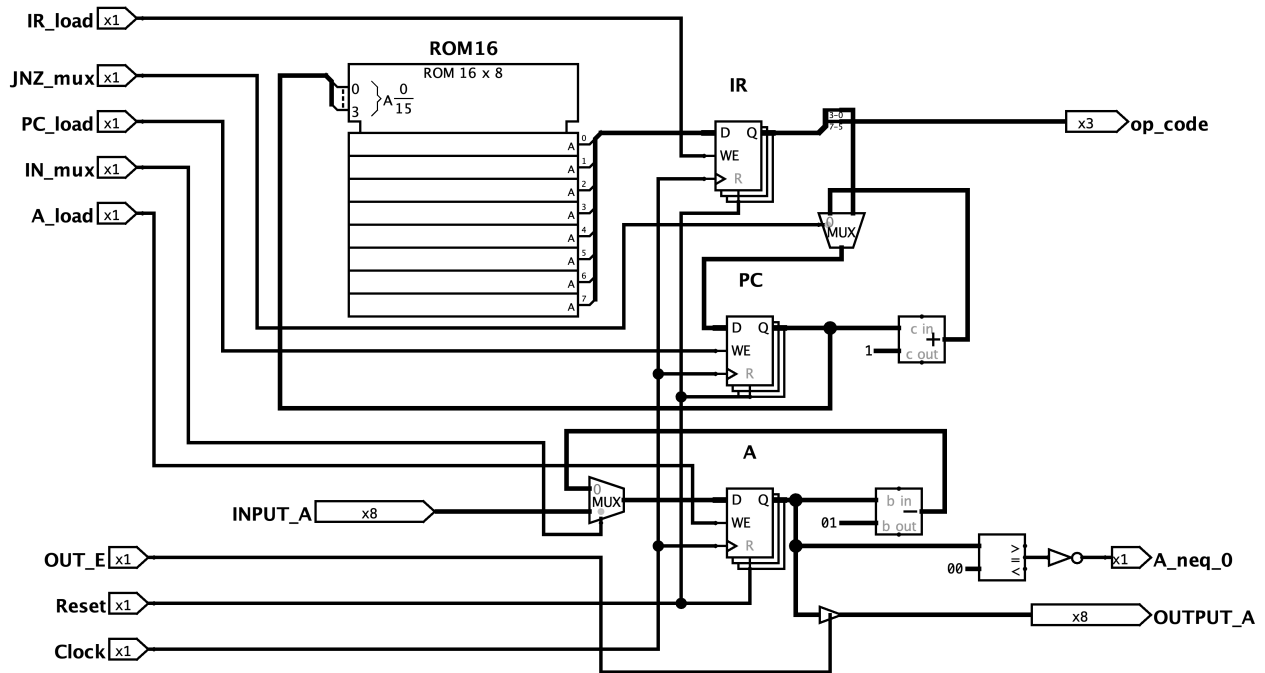


Figure 2: Reference `ec1_datapath` circuit. IR is loaded from ROM16 (addressed by PC) and split into `op_code` (3 bits) and the jump address; `JNZ_mux` selects between PC's own +1 incrementer and that jump address for PC's D input. `IN_mux` selects between `INPUT_A` and the decremented A for A's D input; a comparator against 0 produces `A_neq_0`, and a tri-state buffer gated by `OUT_E` drives `OUTPUT_A`.

Block	Purpose
PC	4-bit register. D comes from JNZmux: input 0 is $PC + 1$ (Incrementer), input 1 is $IR[3:0]$ (jump target). Loads when <code>PCload</code> is asserted. Its output is ROM's <i>only</i> address input.
IR	8-bit register, loaded from ROM's data output when <code>IRload</code> is asserted. $IR[7:5]$ (opcode) goes to the Control Unit; $IR[3:0]$ goes to JNZmux input 1.
ROM	16×8 -bit program memory, addressed directly by PC.
A (Accumulator)	8-bit register. D comes from INmux: input 1 is the external Input port, input 0 is the 8-bit Decrementer's output ($A - 1$). Loads when <code>Aload</code> is asserted.
Zero check	An 8-input OR (or NOR, inverted) over A, producing ($A \neq 0$) for the Control Unit's JNZ test.
Tri-state buffer	Drives A onto Output only when <code>OutE</code> is asserted; high-impedance otherwise.

Quick Experiments (Try It)

Try It 1: PC counts on its own

Build just the 4-bit PC register, the 4-bit Incrementer, and the JNZmux (tie its select to 0 for now). Single-step a manual clock.

- Does PC count 0, 1, 2, ..., 15, 0, ... on its own?
- Set JNZmux's select to 1 and poke $IR[3:0]$ to some value. What happens to PC on the next clock edge?

Try It 2: Decrementer wrap-around

Build the 8-bit Decrementer alone (an 8-bit Adder/Subtractor computing $A - 1$) and poke a few input values with the Poke tool.

1. What does it output for input 1? For input 0?
2. In the actual countdown program, JNZ exits the loop the instant $A = 0$ – so this wrap-around case never executes. What *would* happen if a program kept calling DEC after A reached 0?

Try It 3: A trivial ROM image

Build just PC $\rightarrow$ ROM $\rightarrow$ IR (no Accumulator side yet). Load this image (a HALT at address 0, zeros elsewhere) with *Load Image*:

```
v2.0 raw
e0
```

1. Single-step once. Does IR read E0?
2. Does PC still advance to 1 even though the instruction is HALT? (Whether the machine actually *stops* is a Control Unit job, not a datapath job – you haven't built that yet.)

In-Lab Exercises (target: 3 hours)**In-Lab Exercise 1: PC side (25 min)**

Build the 4-bit PC register, the 4-bit Incrementer, and the JNZmux (2:1, 4-bit) from Try It 1. Wire PC's output directly to where ROM's address input will go – there is no separate address register anywhere in this machine.

In-Lab Exercise 2: Instruction Register (15 min)

Build the 8-bit IR register. Split its output into IR[7:5] (opcode, 3 bits) and IR[3:0] (address, 4 bits) using Logisim's Splitter.

In-Lab Exercise 3: ROM (15 min)

Add a 16×8 -bit **ROM** component and wire it into the PC/IR pair from Exercises 1–2: address from PC, data output to IR. Leave it empty for now – you will load the countdown program in Exercise 7.

In-Lab Exercise 4: Accumulator side (30 min)

Build the 8-bit A register, the 8-bit Decrementer from Try It 2, the INmux (2:1, 8-bit), the Zero-check (an 8-input OR gate; its output *is* ($A \neq 0$) directly), and a Tri-state Buffer on A's output, enabled by OutE.

In-Lab Exercise 5: Design the Control Unit (40 min)

Using the ISA and the fetch/decode/execute cycle, design the FSM yourself:

1. List every state you need. Start with FETCH and DECODE; then decide, for each of the 5 instructions, whether it needs a state of its own. Do the 3 unused opcodes need one?
2. Draw the state diagram: a circle per state, an arrow per transition, labelled with the opcode (or condition) that causes it. Pay attention to JNZ – how many different next-states can it

lead to?

3. Encode your states in a 3-bit register $Q_2Q_1Q_0$. (Hint: if DECODE can transition straight to a state numerically equal to the opcode, your next-state logic gets much smaller.)
4. Derive Boolean equations for D_2, D_1, D_0 (next state) and for every output (IRload, PCload, INmux, Aload, JNZmux, OutE, Halt).

Keep this design on paper – you will check it against Appendix A and then build it in Exercise 6.

In-Lab Exercise 6: Build the Control Unit (35 min)

Build the 3-bit state register (three D flip-flops, or a Logisim Register), the state-decoder gates, and the next-state/output logic directly from *your* Exercise 5 design. Wire ($A \neq 0$) in from the Accumulator side and IR[7:5] in from IR. Check your design against Appendix A before you wire it up if you want a sanity check – Figure 5 there shows a reference circuit, but only look *after* you’ve built your own.

Figure 3 shows how Exercises 1–6 combine into the top-level `ec1_micro` circuit for Exercise 7: `ec1_datapath` and `ec1_control` as two subcircuits, with `Reset` and `Clock` fanning out to both, `INPUT_A` entering the datapath, `OUTPUT_A` driving a hex display, and `HALT` driving an LED.

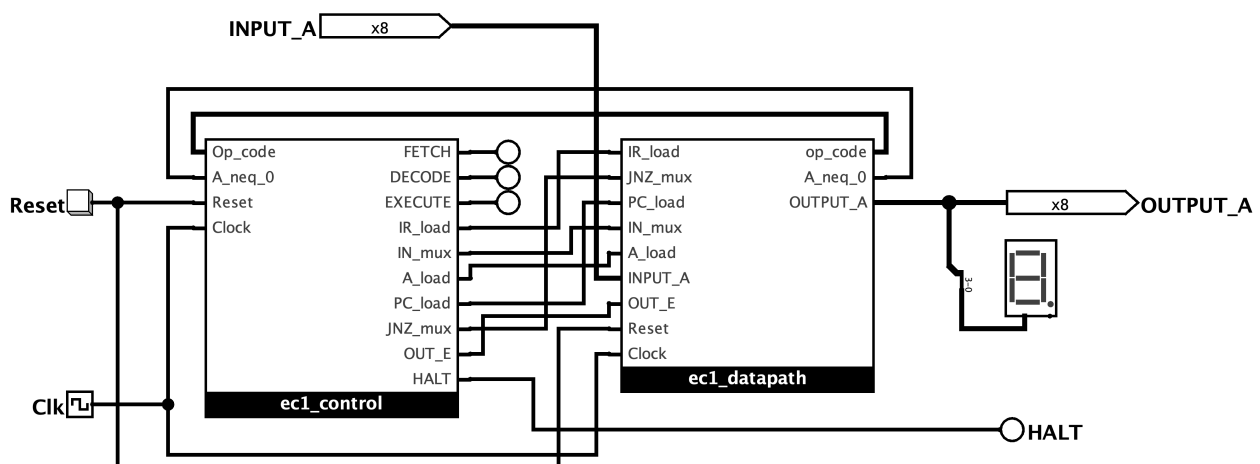


Figure 3: Reference `ec1_micro` top-level circuit: `ec1_control` and `ec1_datapath` wired together with `Reset`, `Clock`, `INPUT_A`, `OUTPUT_A` (with hex display), and `HALT`.

In-Lab Exercise 7: Assemble, load, and run (20 min)

Wire Exercises 1–6 together following Figures 2 and 3. Hand-assemble the countdown program below and load it with ROM’s *Load Image*:

Addr	Hex	Assembly
00	60	IN A ; Input to A
01	80	OUT A ; Output A
02	A0	DEC A ; Decrement A
03	C1	JNZ 01 ; Jump to 01 if A != 0
04	E0	HALT ; Halt execution
05-0F	00	(unused)

Logisim plain-text image (paste into ROM’s *Edit Contents*, or save as `.txt` and *Load Image*):

```
v2.0 raw
```

```
60 80 a0 c1 e0 00 00 00 00 00 00 00 00 00 00
```

Set **Input** to 10 (0A hex) with the Poke tool, then single-step and confirm **Output** shows 10, 9, 8, . . . , 1 in turn, followed by **Halt** going high with the state register no longer changing.

AI Collaboration

Try these prompts with an AI tool:

- “What is the difference between a hardwired control unit and a microprogrammed one?”
- “Given this state transition table [paste yours], derive minimal sum-of-products equations for the next-state logic.”
- “Which Logisim-Evolution component should I use for an 8-input OR reduction, and how do I wire an 8-bit bus into it?”

Critical check – verify, don’t just accept: Ask the AI to derive **PCload** and **Aload** from your state table. Then check *every* row of your table against its equations by hand – not just the common states, but **JNZ** specifically, since its conditional term is the easiest place for a minimization mistake to hide.

Know the limit: AI tools will confidently produce next-state equations that satisfy *most* rows of a state table while silently getting one edge case wrong. It will not necessarily flag which row it’s least sure about – that’s your job.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: The first six clock cycles

Assume ROM holds the countdown program from Exercise 7 and **Input** was poked to 3 before the first **FETCH**. Fill in a table with columns *Cycle*, *State*, *PC*, *IR (hex)*, *A* for the first six clock edges. At which cycle does **Output** first show a value, and what is it?

Find the Bug 1: A one-line control equation

A student built **PCload** = **Fetch** (forgetting the **JNZ** term entirely) but wired everything else correctly. Predict exactly what the countdown program’s **Output** sequence looks like when run on this machine, and explain why it goes wrong where it does.

Write Code on Paper 1: Design a sixth instruction

EC-1 has three unused opcodes. Using 010, design **INC A** ($A \leftarrow A + 1$). Write its encoding, describe any new datapath component or mux input it needs (if any), and write the modified **Aload** and **INmux** equations (or a new control signal) required to support it.

Draw on Paper 1: Extended state diagram

Redraw your Exercise 5 state diagram with **INC** (from the previous exercise) added as an eighth state. Label its transition from **DECODE** and its return to **FETCH**. Compare against your original diagram: how many new states, gates, and wires did one extra instruction actually

cost?

Reflection

In one short paragraph, compare EC-1's control unit to Lab 1's control-word machine. Both machines execute a sequence of "instructions" stored in ROM and both can jump conditionally – but EC-1 has an Instruction Register, an opcode field, and a multi-state hardwired FSM, while Lab 1's machine has neither. What did EC-1's FSM *buy* you that Lab 1's machine does not have? What did Lab 1's machine give up to avoid needing one?

Appendix A: Reference Control-Unit Design (Self-Check)

Use this *after* attempting Exercise 5 – not before.

$Q_2Q_1Q_0$	State	What happens
000	FETCH	$IR \leftarrow ROM[PC], PC \leftarrow PC + 1.$
001	DECODE	Inspect $IR[7:5]$; no datapath signals asserted.
011	INPUT	$A \leftarrow \text{Input}.$
100	OUTPUT	$\text{Output} \leftarrow A.$
101	DEC	$A \leftarrow A - 1.$
110	JNZ	If $A \neq 0$: $PC \leftarrow IR[3:0]$; either way, return to FETCH.
111	HALT	Stop (self-loop).

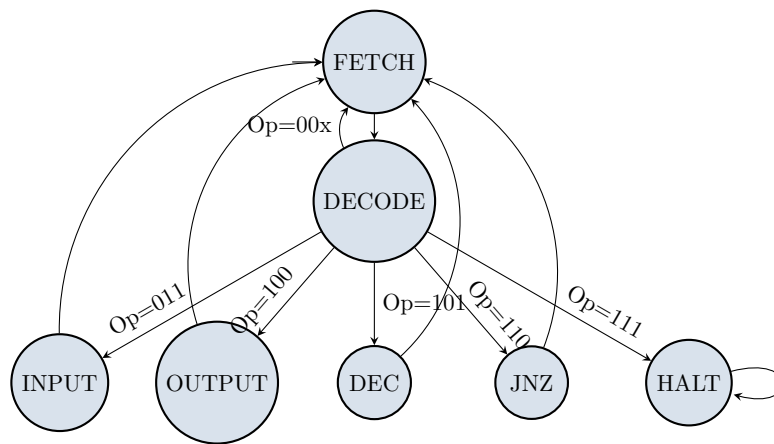


Figure 4: EC-1 Control Unit FSM (Lecture 10, Figure 10-3).

Using $\text{Fetch} = \bar{Q}_2\bar{Q}_1\bar{Q}_0$, $\text{Decode} = \bar{Q}_2\bar{Q}_1Q_0$, $\text{Input} = \bar{Q}_2Q_1\bar{Q}_0$, $\text{Output} = Q_2\bar{Q}_1\bar{Q}_0$, $\text{Dec} = Q_2\bar{Q}_1Q_0$, $\text{JNZ} = Q_2Q_1\bar{Q}_0$, $\text{Halt} = Q_2Q_1Q_0$, and $O_2O_1O_0 = IR[7:5]$:

Next state	Equation
D_2	$= \text{Decode} \cdot O_2 + \text{Halt}$
D_1	$= \text{Decode} \cdot O_1 \cdot (O_2 + O_0) + \text{Halt}$
D_0	$= \text{Fetch} + \text{Decode} \cdot O_0 \cdot (O_2 + O_1) + \text{Halt}$

Output	Equation
$IRload$	$= \text{Fetch}$
$PCload$	$= \text{Fetch} + \text{JNZ} \cdot (A \neq 0)$
$INmux$	$= \text{Input}$
$Aload$	$= \text{Input} + \text{Dec}$
$JNZmux$	$= \text{JNZ}$
$OutE$	$= \text{Output}$
$Halt$	$= \text{Halt}$

Figure 5 is a reference `ec1_control` circuit implementing the design above – one way to build it, not the only way. Compare it against your own next-state and output equations only *after* you’ve derived them yourself.

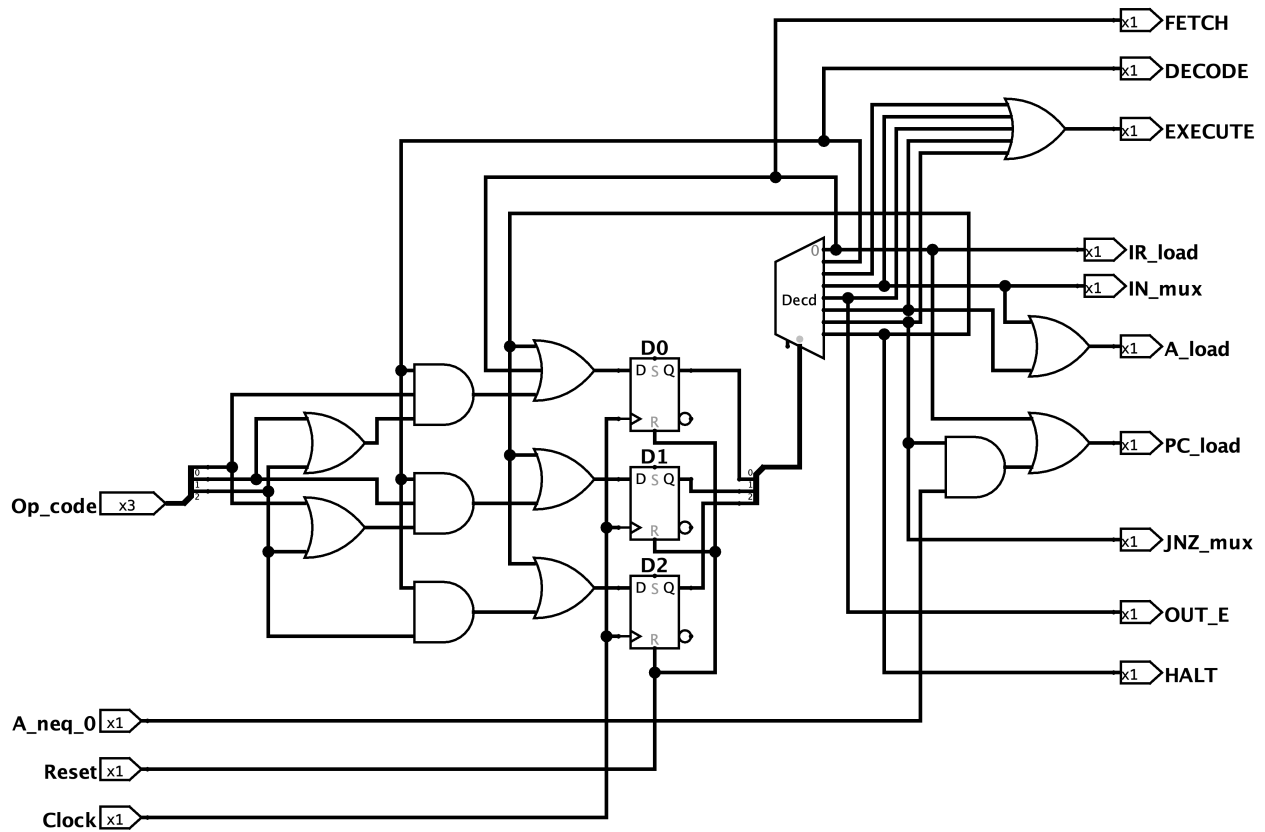


Figure 5: Reference `ec1_control` circuit. `Op_code` feeds the next-state logic for the D0, D1, D2 state flip-flops (with `Reset` and `Clock`); the resulting 3-bit state, together with ($A \neq 0$), drives a decoder/mux (`Decd`) that asserts `IR_load`, `IN_mux`, `A_load`, `PC_load`, `JNZ_mux`, `OUT_E`, and `HALT` in the correct states, plus the `FETCH/DECODE/EXECUTE` status LEDs.